

Entrega 1

1. Introducción

Este documento presenta el diseño de la base de datos relacional para Wheels Alpes, la plataforma de carpooling universitario descrita en el caso de estudio. Se parte de los requerimientos de negocio para derivar un modelo conceptual E/R, se transforma a un modelo relacional mediante el algoritmo de Chen, se lleva el modelo a la Forma Normal de Boyce-Codd (FNBC) y se diseñan los escenarios de prueba que validan la integridad de los datos y las reglas de negocio.

2. Modelo conceptual (E/R)

Del análisis del caso se identifican seis entidades. El modelo E/R completo se adjunta en formato fuente de Oracle Data Modeler en la carpeta docs/.

2.1 Entidades

- **Estudiante** — persona registrada en la plataforma; puede actuar como conductor, pasajero o ambos.
- **Vehículo** — automóvil inscrito por un conductor para publicar viajes.
- **Viaje** — trayecto publicado por un conductor en un vehículo, con fecha, hora, ruta, tarifa y estado.
- **Parada** — punto de recogida asociado a un viaje; pertenece a una localidad.
- **Solicitud** — petición de un pasajero para un cupo en un viaje, eligiendo una parada de recogida.
- **Localidad** — zona geográfica usada para agrupar las paradas (base de las consultas RFC1 y RFC2).

2.2 Rol de usuario

El caso distingue conductores y pasajeros, pero no les asigna atributos distintos: ambos se describen con los mismos datos personales. Por eso se modela una sola entidad Estudiante con un atributo booleano `es_conductor`, en lugar de una jerarquía de subtipos. Esto evita tablas adicionales sin ganancia de integridad y permite que un mismo estudiante sea conductor y pasajero a la vez.

2.3 Relaciones y cardinalidades

Relación	Cardinalidad	Regla de negocio
Estudiante — Vehículo	1 : N	Un conductor puede inscribir varios vehículos; cada vehículo pertenece a un conductor (RF2).
Vehículo — Viaje	1 : N	Un viaje se publica con un vehículo; un vehículo puede usarse en varios viajes (RF3).
Viaje — Parada	1 : N	Cada viaje define su propia lista de paradas; una parada pertenece a un viaje.
Localidad — Parada	1 : N	Cada parada está en una localidad; una localidad agrupa varias paradas.
Estudiante — Solicitud	1 : N	Un pasajero genera varias solicitudes; cada solicitud es de un pasajero (RF10).
Viaje — Solicitud	1 : N	Un viaje recibe varias solicitudes; cada solicitud apunta a un viaje.
Parada — Solicitud	1 : N	El pasajero elige una parada de recogida por solicitud (RF10).

3. Modelo relacional (algoritmo de Chen)

El modelo conceptual se transforma a relacional aplicando el algoritmo de Chen. La documentación completa (atributos, dominios, restricciones y tuplas de ejemplo) está en la pestaña 1. Chen de la plantilla de Excel. A continuación se justifica el origen de cada tabla.

Tabla	Justificación de la transformación
Estudiante	Entidad fuerte → tabla. PK = documento_identidad. Todos los atributos dependen directamente de la llave.
Vehiculo	Entidad fuerte → tabla. PK = placa. La relación 1:N con Estudiante se implementa con la FK documento_conductor.
Viaje	Entidad fuerte → tabla. PK = id_viaje (sustituta). La relación 1:N con Vehículo se implementa con la FK placa_vehiculo. El atributo estado se restringe con un CHECK.
Parada	Entidad fuerte → tabla. PK = id_parada. Recibe dos FK: id_viaje e id_localidad, por sus dos relaciones 1:N.
Localidad	Entidad fuerte → tabla. PK = id_localidad. Se modela como entidad y no como texto en Parada para permitir agrupar y filtrar por localidad de forma consistente (RFC1, RFC2).
Solicitud	Resuelve la relación entre Estudiante (pasajero), Viaje y Parada. Tiene atributos propios (fecha, estado, fecha_cancelacion), por lo que se modela como entidad con PK id_solicitud y tres FK.

4. Normalización (hasta FNBC)

Se verificó cada relación contra 1FN, 2FN, 3FN y FNBC. El detalle está en la pestaña 2. Normalización del Excel. Cinco de las seis relaciones ya estaban en FNBC; la relación Viaje requirió transformación.

4.1 Verificación por relación

Relación	3FN	FNBC	¿Transforma?	Motivo
Estudiante	Cumple	Cumple	No	PK simple; atributos dependen directamente de ella.
Vehículo	Cumple	Cumple	No	PK simple; sin dependencias transitivas.
Viaje	No cumple	No cumple	Sí	Dependencia transitiva placa_vehiculo → documento_conductor.
Solicitud	Cumple	Cumple	No	PK simple; sin dependencias transitivas.
Parada	Cumple	Cumple	No	PK simple; sin dependencias transitivas.
Localidad	Cumple	Cumple	No	PK simple; sin dependencias transitivas.

4.2 Transformación de la relación Viaje

La versión inicial de Viaje incluía placa_vehiculo y documento_conductor. Existe una dependencia transitiva:

id_viaje → placa_vehiculo → documento_conductor

El conductor no depende directamente del viaje sino del vehículo, y placa_vehiculo no es llave candidata de Viaje, por lo que la relación viola 3FN y FNBC. Se elimina documento_conductor de Viaje; el conductor se obtiene consultando Vehiculo a través de placa_vehiculo. Con esto la relación queda en FNBC. El modelo relacional final en FNBC está en la sección 3 de la pestaña de normalización.

5. Escenarios de prueba

Los escenarios se dividen en dos grupos: pruebas de funcionalidad (validan cada requerimiento sobre el modelo) y pruebas de corrección y calidad de los datos (unicidad, integridad referencial, restricciones de chequeo y reglas de negocio, con casos de éxito y de falla).

5.1 Pruebas de funcionalidad (RF, RFC, RFA)

Req.	Escenario	Datos de entrada	Resultado esperado
RF1	Registrar un estudiante como conductor	Documento 1021928371; datos personales; es_conductor = 1	El estudiante queda registrado como conductor.
RF2	Inscribir un vehículo a un conductor	Placa XYZ793; Toyota Corolla 2023; capacidad 4; conductor 1021928371	El vehículo queda registrado y asociado al conductor.
RF3	Publicar un viaje con el vehículo inscrito	id_viaje de ejemplo + vehículo XYZ793 + fecha, hora, origen, ruta, cupos, costo, estado	El viaje queda registrado y asociado al vehículo.
RF4	Aceptar una solicitud de un viaje	Solicitud existente con su id_viaje e id_parada	La solicitud pasa a aceptada, asociada al viaje y la parada.
RF5	Avanzar el estado del viaje	Viaje en estado programado	El viaje pasa a en_curso y luego a completado.
RF6	Listar los viajes de un conductor	Conductor 1021928371, vehículo XYZ793	Se muestran los viajes con su estado y pasajeros confirmados.
RF7	Registrar un estudiante como pasajero	documento_pasajero presente en Solicitud con sus datos	El pasajero queda registrado en Estudiante.
RF8	Actualizar datos de un pasajero	Pasajero existente; se modifica el celular	Los datos se actualizan sin alterar el documento.
RF9	Buscar viajes por fecha y localidad	Fecha de un viaje existente + localidad 0001 (Chapinero)	Se muestran los viajes programados que coinciden.
RF10	Crear una solicitud de cupo	documento_pasajero + id_viaje + id_parada existentes	Se crea la solicitud asociada al pasajero, viaje y parada.
RF11	Cancelar con más de 2 h de anticipación	Solicitud existente; faltan más de 2 h para la salida	La solicitud pasa a cancelada y se registra fecha_cancelacion.
RFC1	Contar solicitudes por localidad	Localidades 0001 y 0002 sobre solicitudes y paradas	Se obtiene la cantidad de solicitudes por localidad.
RFC2	Estimar ingresos por localidad	Viajes, sus costos y localidades vía paradas	Se obtiene el ingreso estimado por localidad.
RFC3	Listar viajes de un conductor en el tiempo	Conductor 1021928371 + viajes asociados	Se muestran los viajes del conductor según los filtros.
RFA1	Estadísticas del conductor	Conductor 1021928371 y sus viajes completados	Total de pasajeros, tiempo, ingresos y distancia.
RFA2	Estadísticas del pasajero	Pasajero con solicitudes aceptadas y viajes completados	Número de viajes, zona más usada, gasto total, distancia y tiempo promedio.

5.2 Pruebas de corrección y calidad de los datos

Tipo de prueba	Escenario	Datos de entrada	Resultado esperado
Unicidad PK — éxito	Registrar un estudiante con PK nueva	documento_identidad = 1021928372 + datos	Se registra correctamente.

Tipo de prueba	Escenario	Datos de entrada	Resultado esperado
Unicidad PK — falla	Registrar otro estudiante con un documento existente	documento_identidad = 1021928372 (repetido)	Rechazado: la PK debe ser única.
Integridad FK — éxito	Inscribir un vehículo a un conductor existente	placa = XYZ793, documento_conductor = 1021928371	Se registra: el conductor existe en Estudiante.
Integridad FK — falla	Inscribir un vehículo a un conductor inexistente	placa = ABC123, documento_conductor = 9999999999	Rechazado: la FK no existe en Estudiante.
CHECK — éxito	Registrar un vehículo con capacidad válida	capacidad_cupos = 4	Se registra: cumple CK(1–6).
CHECK — falla	Registrar un vehículo con capacidad fuera de rango	capacidad_cupos = 8	Rechazado: viola CK(1–6).
Regla de negocio — falla	Cancelar una solicitud con menos de 2 h	Solicitud aceptada; falta 1 h para la salida	Rechazado: la cancelación exige mínimo 2 h (RF11).
Rol de usuario — falla	Un pasajero intenta aceptar una solicitud	Usuario con rol de pasajero	Rechazado: aceptar/rechazar corresponde al conductor (RF4).

Referencias

Para la corrección de errores de redacción y gramática de este documento se utilizó la herramienta de inteligencia artificial generativa Claude Sonnet 5 (Anthropic), con la instrucción: "Arregla los errores de redacción y gramática". Los autores revisaron y validaron el resultado, y asumen la responsabilidad total sobre el contenido.